

Healthcare Insurance Analytics Project

Project Implementation Evidence and Screenshot Documentation

Document Prepared by:
Kusum Katwal

1. Project Execution Summary

This document walks through the build in the order I actually did it — setting up tracking and documentation first, then moving data into S3, cleaning it in Databricks, loading it into Redshift, and finally generating the 13 business output tables. Each section below pairs a short explanation with the screenshot that proves that step happened.

Phase	Tool / Service	Main Activity	Evidence Captured
Planning	Jira	Created Scrum board and tracked project tasks	Jira board screenshots
Documentation	Google Docs / Word	Prepared SRS, Solution Design, and Database Schema Design documents	Document cover screenshots
Ingestion	AWS S3	Created bucket and uploaded input datasets	S3 bucket and input-data screenshots
Transformation	Databricks + PySpark	Read raw files, created Bronze layer, checked nulls/duplicates, cleaned Silver datasets	Notebook execution screenshots
Storage and Analytics	AWS Redshift	Created schemas/tables, loaded Silver data, created output tables	Redshift Query Editor screenshots
Completion	Jira	Moved all project tasks to Done	Final Jira dashboard screenshot

Step 1: Jira Project Setup

- A Scrum board was created for this project with To Do, In Progress, and Done columns, so every task — documentation, ingestion, cleaning, loading, and output generation — has a visible home.

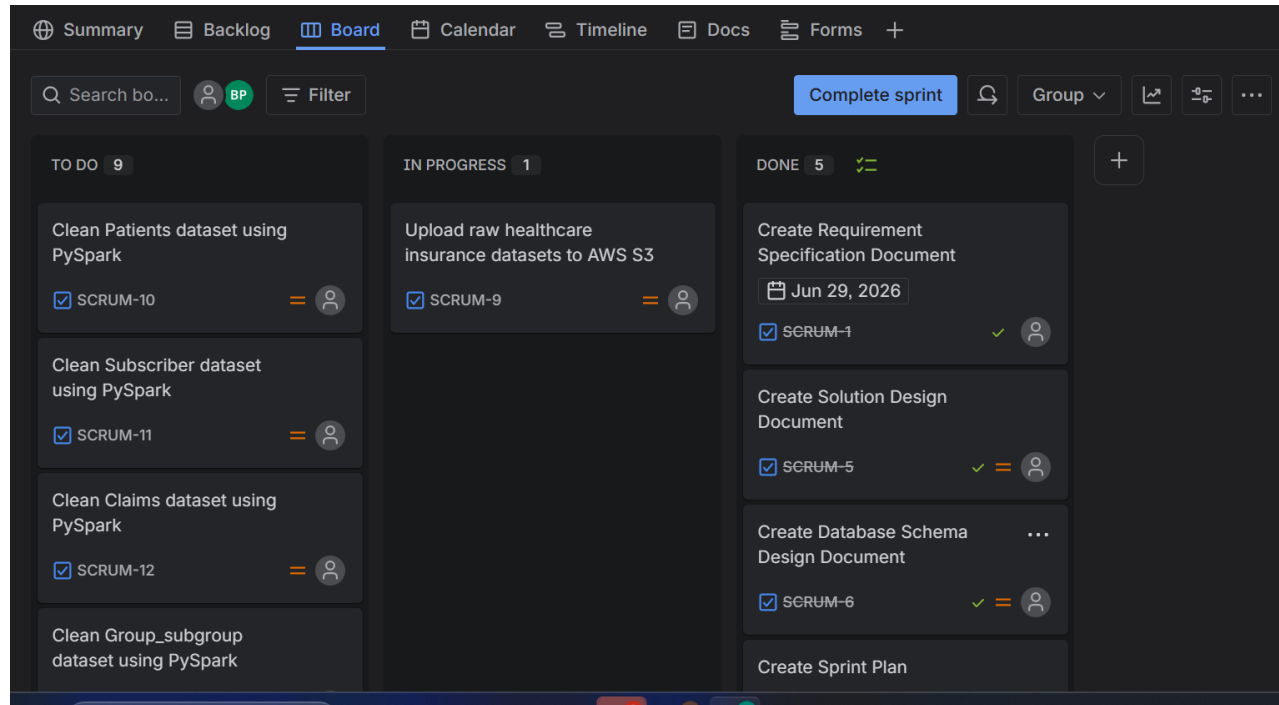


Figure 1. Jira-Project Setup

Step 2: Requirement Specification Document Created

- The SRS was written first, covering purpose, scope, the 13 business use cases, functional and non-functional requirements, and acceptance criteria.

Step 3: Solution Design Document Created

- The Solution Design document maps out the architecture, technology choices, data flow, and Redshift schema before any code was written.

Step 4: Database Schema Design Document Created

- The target Redshift tables, their key columns, and the primary/foreign key relationships were laid out so the cleaning and loading steps have a clear target.

Step 5: AWS S3 Bucket Created

- An S3 bucket was created to act as the single landing zone for raw data central storage location for input, Bronze, and Silver data layers.

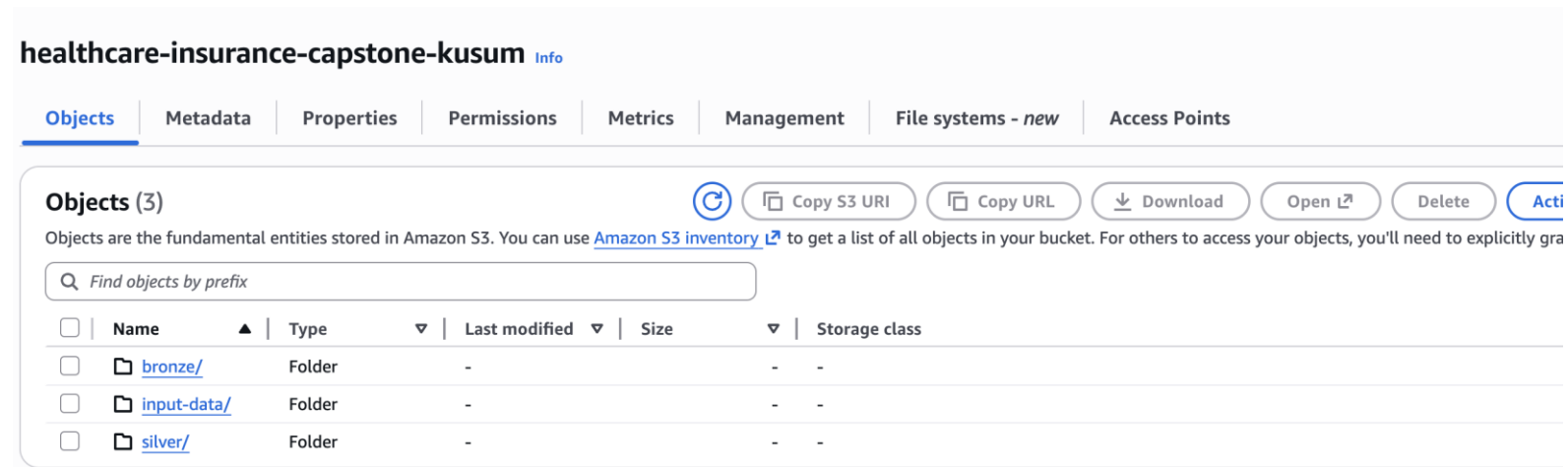
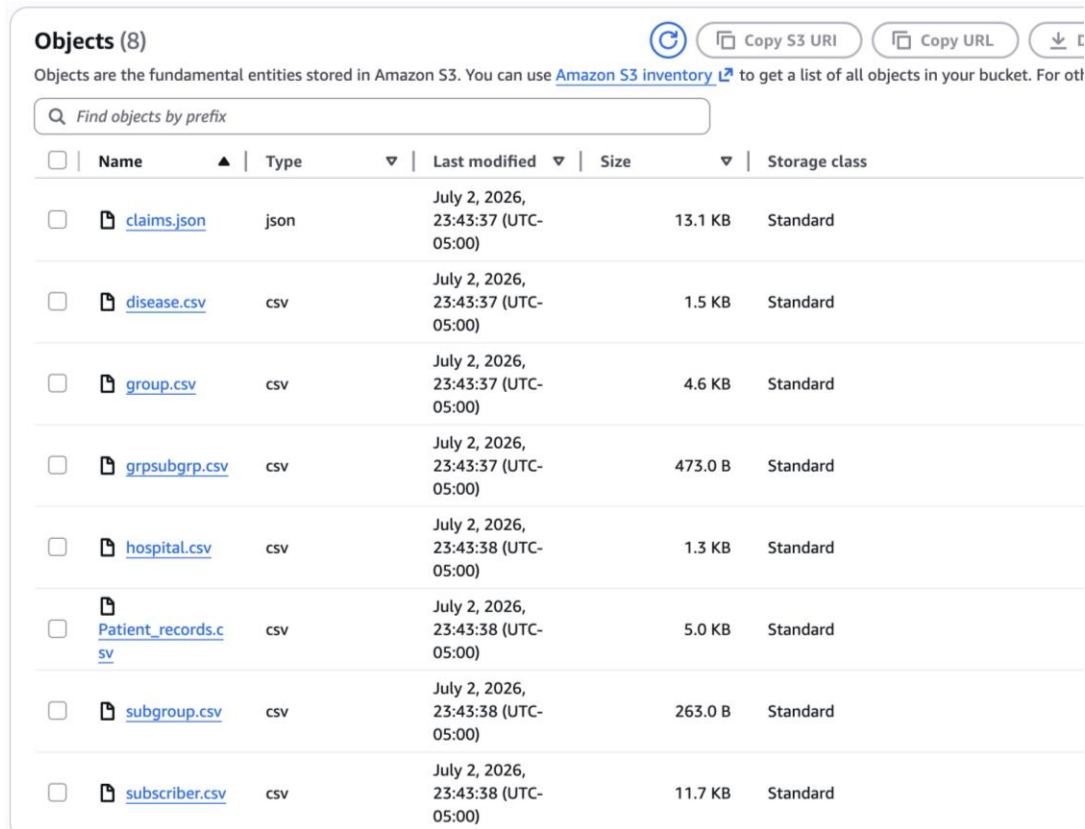


Figure 5. AWS S3 Bucket Created

Step 6: Raw Datasets Uploaded to S3 input-data Folder

- The four raw datasets — Patients, Subscriber, Claims, and Group_subgroup — were uploaded into an input-data folder inside the bucket.



Objects (8)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For other actions, see [Amazon S3 console actions](#).

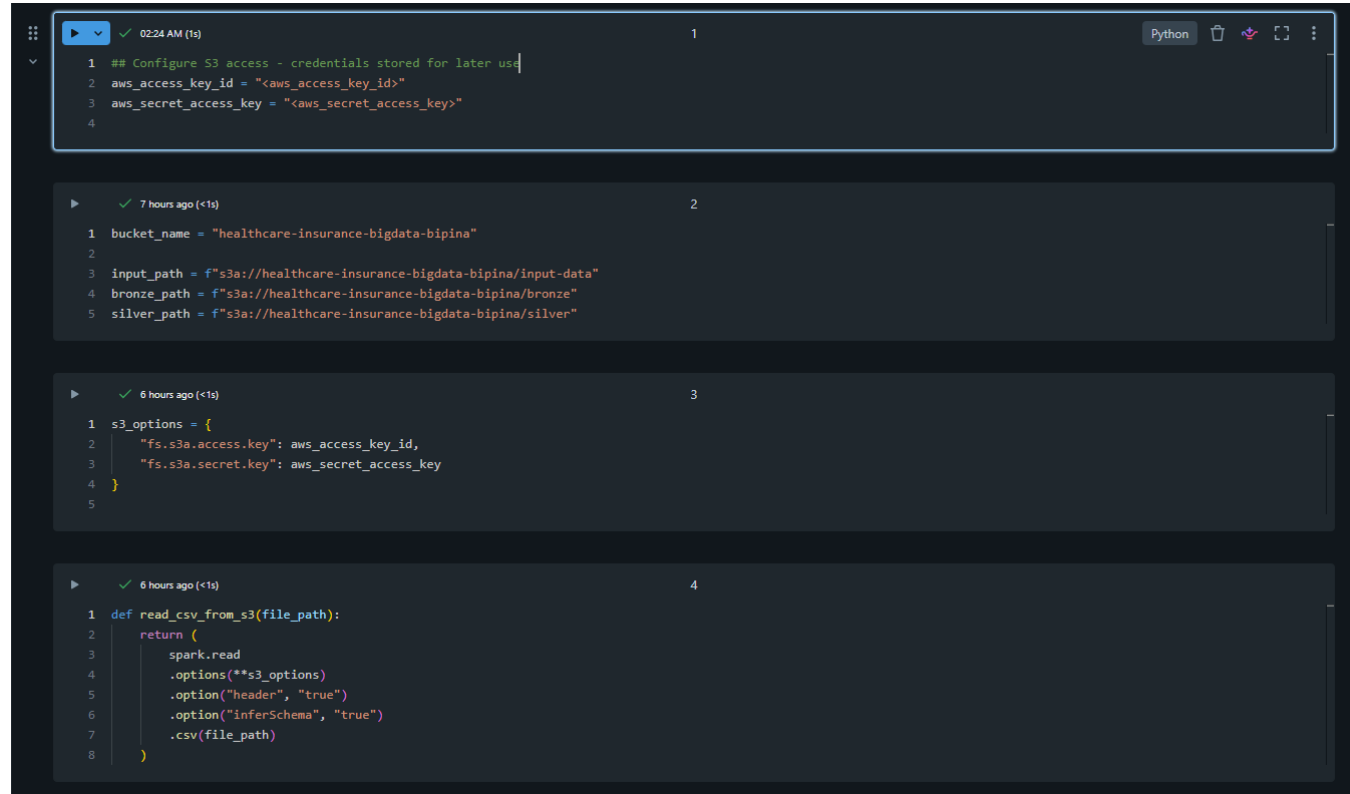
Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	claims.json	json	July 2, 2026, 23:43:37 (UTC-05:00)	13.1 KB	Standard
☐	disease.csv	csv	July 2, 2026, 23:43:37 (UTC-05:00)	1.5 KB	Standard
☐	group.csv	csv	July 2, 2026, 23:43:37 (UTC-05:00)	4.6 KB	Standard
☐	grpsubgrp.csv	csv	July 2, 2026, 23:43:37 (UTC-05:00)	473.0 B	Standard
☐	hospital.csv	csv	July 2, 2026, 23:43:38 (UTC-05:00)	1.3 KB	Standard
☐	Patient_records.csv	csv	July 2, 2026, 23:43:38 (UTC-05:00)	5.0 KB	Standard
☐	subgroup.csv	csv	July 2, 2026, 23:43:38 (UTC-05:00)	263.0 B	Standard
☐	subscriber.csv	csv	July 2, 2026, 23:43:38 (UTC-05:00)	11.7 KB	Standard

Figure 6. Raw Datasets Uploaded to S3 input-data Folder

Step 7: Databricks S3 Access Configuration:

- Databricks was configured with the access needed to read from and write back to the S3 bucket, so the notebook can pull the raw files directly.



```
1 ## Configure S3 access - credentials stored for later use
2 aws_access_key_id = "<aws_access_key_id>"
3 aws_secret_access_key = "<aws_secret_access_key>"
4

1 bucket_name = "healthcare-insurance-bigdata-bipina"
2
3 input_path = f"s3a://healthcare-insurance-bigdata-bipina/input-data"
4 bronze_path = f"s3a://healthcare-insurance-bigdata-bipina/bronze"
5 silver_path = f"s3a://healthcare-insurance-bigdata-bipina/silver"

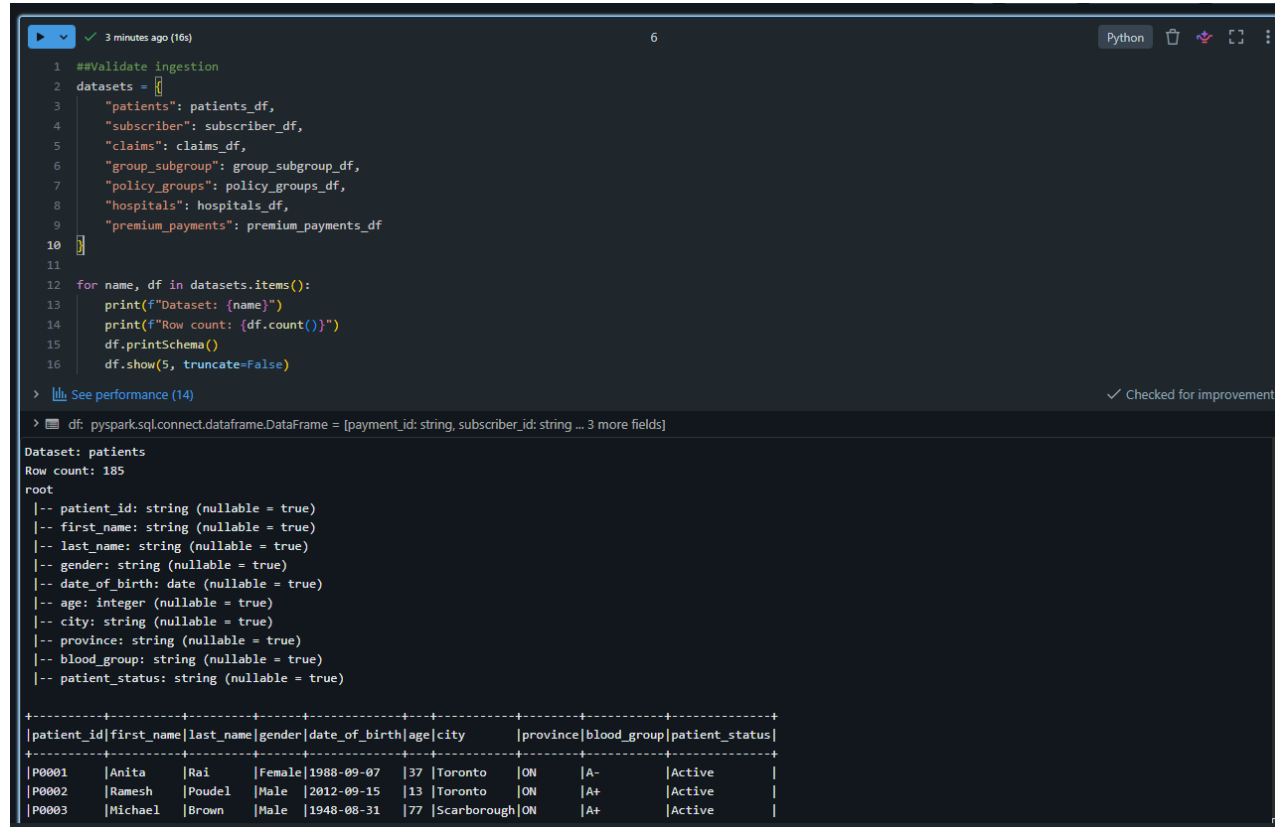
1 s3_options = {
2     "fs.s3a.access.key": aws_access_key_id,
3     "fs.s3a.secret.key": aws_secret_access_key
4 }
5

1 def read_csv_from_s3(file_path):
2     return (
3         spark.read
4         .options(**s3_options)
5         .option("header", "true")
6         .option("inferSchema", "true")
7         .csv(file_path)
8     )
```

Figure 7. Databricks S3 Access Configuration

Step 8: Raw CSV Files Read into S3:

- Each raw CSV was read into a PySpark DataFrame directly from S3, and the schema was printed to confirm the columns came through correctly.



```
1 ##Validate ingestion
2 datasets = [
3     "patients": patients_df,
4     "subscriber": subscriber_df,
5     "claims": claims_df,
6     "group_subgroup": group_subgroup_df,
7     "policy_groups": policy_groups_df,
8     "hospitals": hospitals_df,
9     "premium_payments": premium_payments_df
10 ]
11
12 for name, df in datasets.items():
13     print(f"Dataset: {name}")
14     print(f"Row count: {df.count()}")
15     df.printSchema()
16     df.show(5, truncate=False)
```

> [See performance \(14\)](#) ✓ Checked for improvements

> df: pyspark.sql.connect.dataframe.DataFrame = [payment_id: string, subscriber_id: string ... 3 more fields]

Dataset: patients
Row count: 185

root

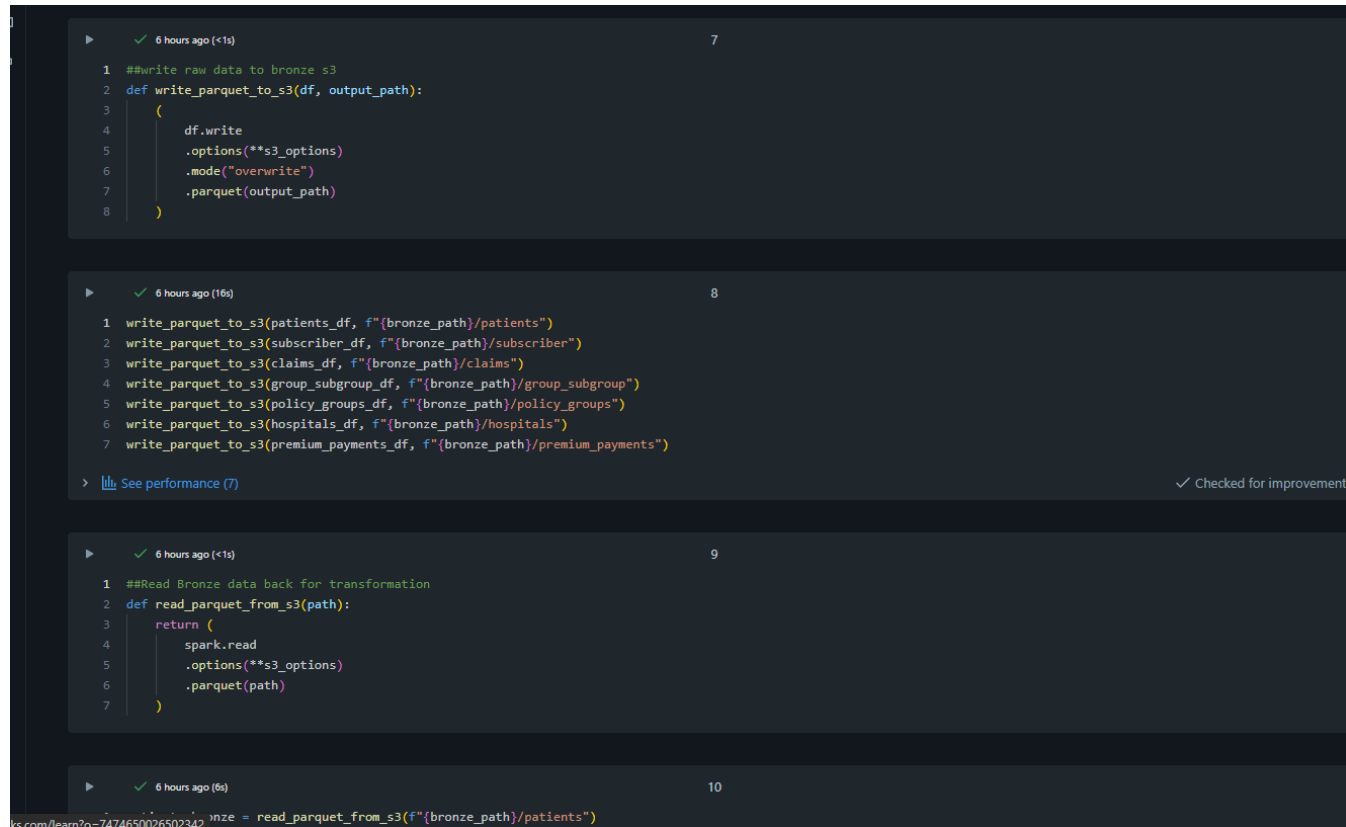
```
-- patient_id: string (nullable = true)
-- first_name: string (nullable = true)
-- last_name: string (nullable = true)
-- gender: string (nullable = true)
-- date_of_birth: date (nullable = true)
-- age: integer (nullable = true)
-- city: string (nullable = true)
-- province: string (nullable = true)
-- blood_group: string (nullable = true)
-- patient_status: string (nullable = true)
```

patient_id	first_name	last_name	gender	date_of_birth	age	city	province	blood_group	patient_status
P0001	Anita	Rai	Female	1988-09-07	37	Toronto	ON	A-	Active
P0002	Ramesh	Poudel	Male	2012-09-15	13	Toronto	ON	A+	Active
P0003	Michael	Brown	Male	1948-08-31	77	Scarborough	ON	A+	Active

Figure 8. Raw CSV Files Read from S3

Step 9: Raw Data Written to Bronze Layer

- After ingestion, raw datasets were written to the Bronze layer in S3. The Bronze layer preserves the original ingested structure and supports traceability before cleaning.



```
▶ 6 hours ago (<1s) 7
1 ##write raw data to bronze s3
2 def write_parquet_to_s3(df, output_path):
3     (
4         df.write
5         .options(**s3_options)
6         .mode("overwrite")
7         .parquet(output_path)
8     )

▶ 6 hours ago (16s) 8
1 write_parquet_to_s3(patients_df, f"{bronze_path}/patients")
2 write_parquet_to_s3(subscriber_df, f"{bronze_path}/subscriber")
3 write_parquet_to_s3(claims_df, f"{bronze_path}/claims")
4 write_parquet_to_s3(group_subgroup_df, f"{bronze_path}/group_subgroup")
5 write_parquet_to_s3(policy_groups_df, f"{bronze_path}/policy_groups")
6 write_parquet_to_s3(hospitals_df, f"{bronze_path}/hospitals")
7 write_parquet_to_s3(premium_payments_df, f"{bronze_path}/premium_payments")
> See performance \(7\) ✓ Checked for improvements

▶ 6 hours ago (<1s) 9
1 ##Read Bronze data back for transformation
2 def read_parquet_from_s3(path):
3     return (
4         spark.read
5         .options(**s3_options)
6         .parquet(path)
7     )

▶ 6 hours ago (6s) 10
1 bronze = read_parquet_from_s3(f"{bronze_path}/patients")
```

Figure 9. Raw Data Written to Bronze Layer

Step 10: Bronze Layer Verified in S3

- The Bronze layer was verified in the S3 console. Separate folders were created for each dataset, such as patients, claims, hospitals, policy_groups, and group_subgroup.

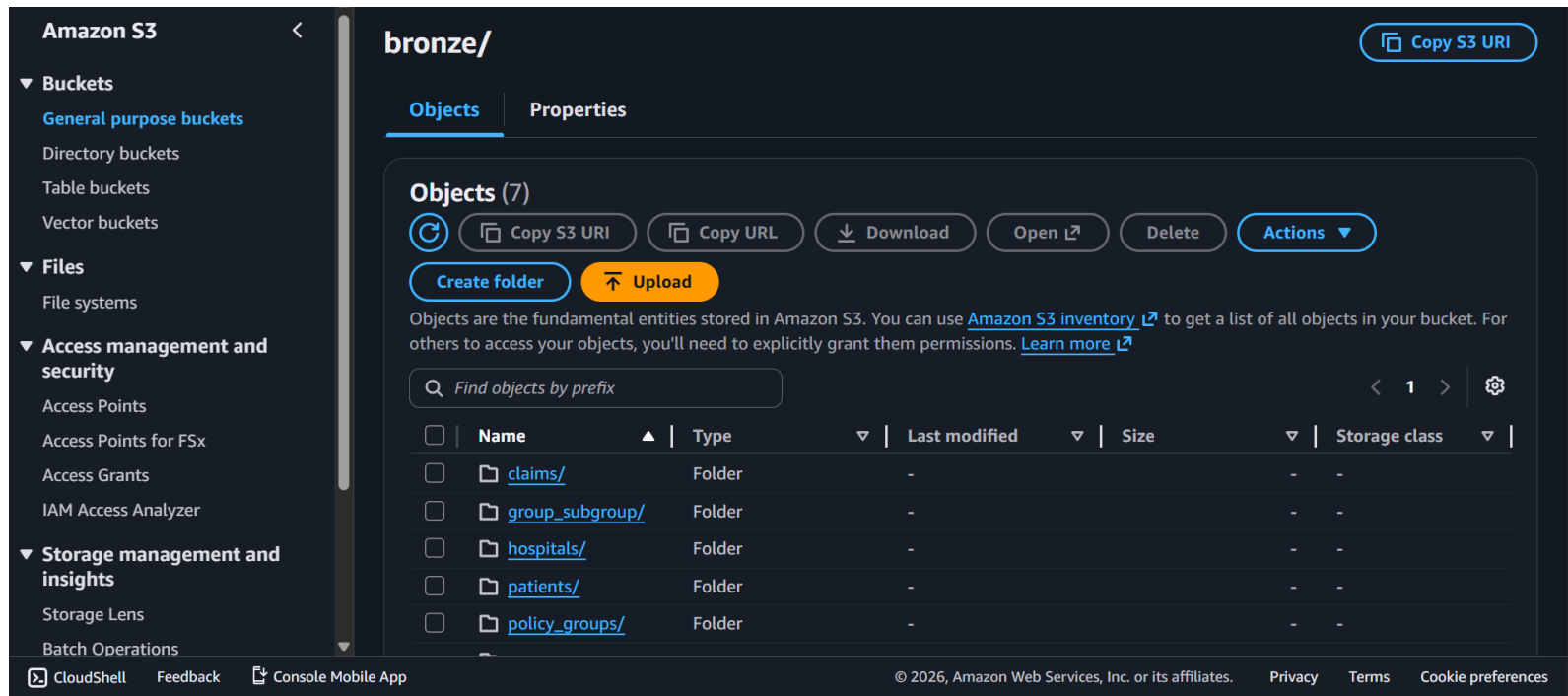


Figure 10. Bronze Layer Verified in S3

Step 11: Null Value Checks Performed

- Before cleaning anything, I counted how many nulls each column had, so I knew exactly what the cleaning step needed to fix.

```
▶ ✓ 2 minutes ago (<1s) 11

1 ##Check null values
2 from pyspark.sql import functions as F
3
4 def null_count_report(df, dataset_name):
5     print(f"Null count report for: {dataset_name}")
6     df.select([
7         F.count(F.when(F.col(c).isNull(), c)).alias(c)
8         for c in df.columns
9     ]).show(truncate=False)

▶ ✓ 1 minute ago (5s) 12

1 null_count_report(patients_bronze, "patients_bronze")
2 null_count_report(subscriber_bronze, "subscriber_bronze")
3 null_count_report(claims_bronze, "claims_bronze")
4 null_count_report(group_subgroup_bronze, "group_subgroup_bronze")

> See performance \(4\) ✓ Checked for improvement

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|patient_id|first_name|last_name|gender|date_of_birth|age|city|province|blood_group|patient_status|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|0|0|0|0|0|0|1|0|1|0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

Null count report for: subscriber_bronze
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|subscriber_id|patient_id|subgroup_id|enrollment_date|premium_frequency|subscription_status|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|0|0|0|0|1|0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

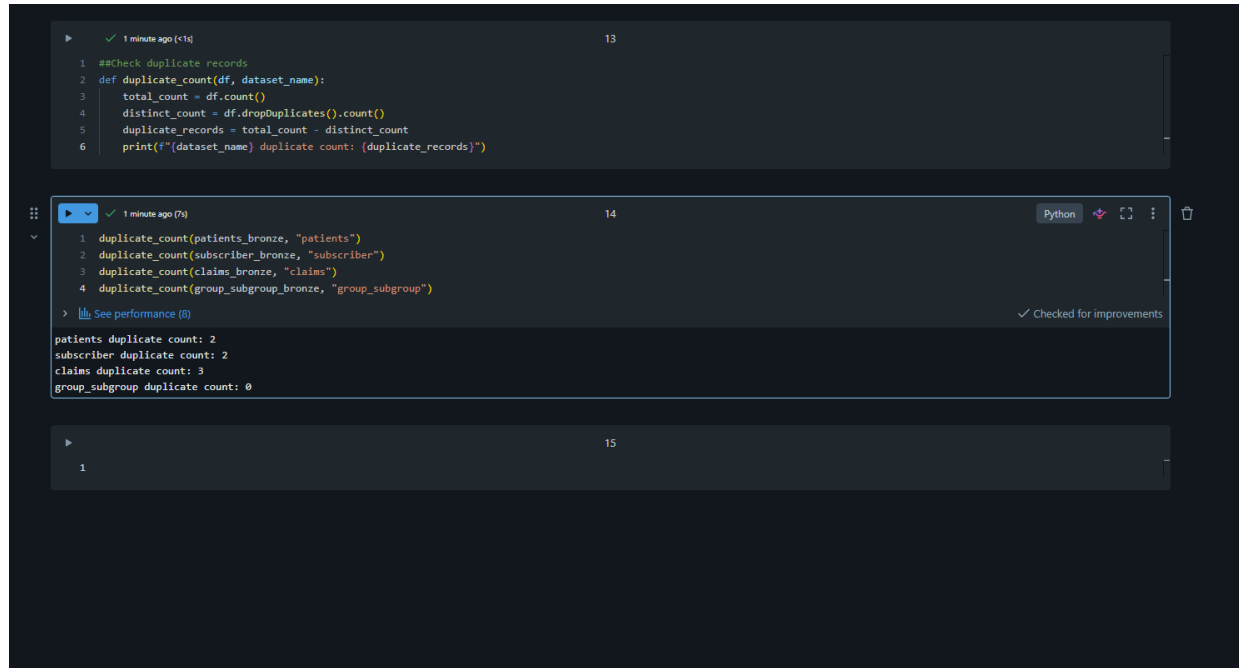
Null count report for: claims_bronze
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|claim_id|subscriber_id|patient_id|hospital_id|disease_name|claim_date|claim_type|total_charges|claim_status|approved_amount|severity|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|0|0|0|0|0|0|1|1|0|0|0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 11. Null Value Checks Performed

Step 12: Duplicate Record Checks Performed

- A duplicate count was run against each dataset to see whether the same record appeared more than once before I removed anything.



The screenshot displays a Jupyter Notebook interface with three visible cells. The top cell (13) defines a function `duplicate_count(df, dataset_name)` that calculates the number of duplicate records by subtracting the distinct count from the total count. The middle cell (14) calls this function for four datasets: `patients_bronze`, `subscriber_bronze`, `claims_bronze`, and `group_subgroup_bronze`. The bottom cell (15) shows the output of these checks.

```
13
1 ##Check duplicate records
2 def duplicate_count(df, dataset_name):
3     total_count = df.count()
4     distinct_count = df.dropDuplicates().count()
5     duplicate_records = total_count - distinct_count
6     print(f"{dataset_name} duplicate count: {duplicate_records}")

14
1 duplicate_count(patients_bronze, "patients")
2 duplicate_count(subscriber_bronze, "subscriber")
3 duplicate_count(claims_bronze, "claims")
4 duplicate_count(group_subgroup_bronze, "group_subgroup")

> See performance \(8\) ✓ Checked for improvements

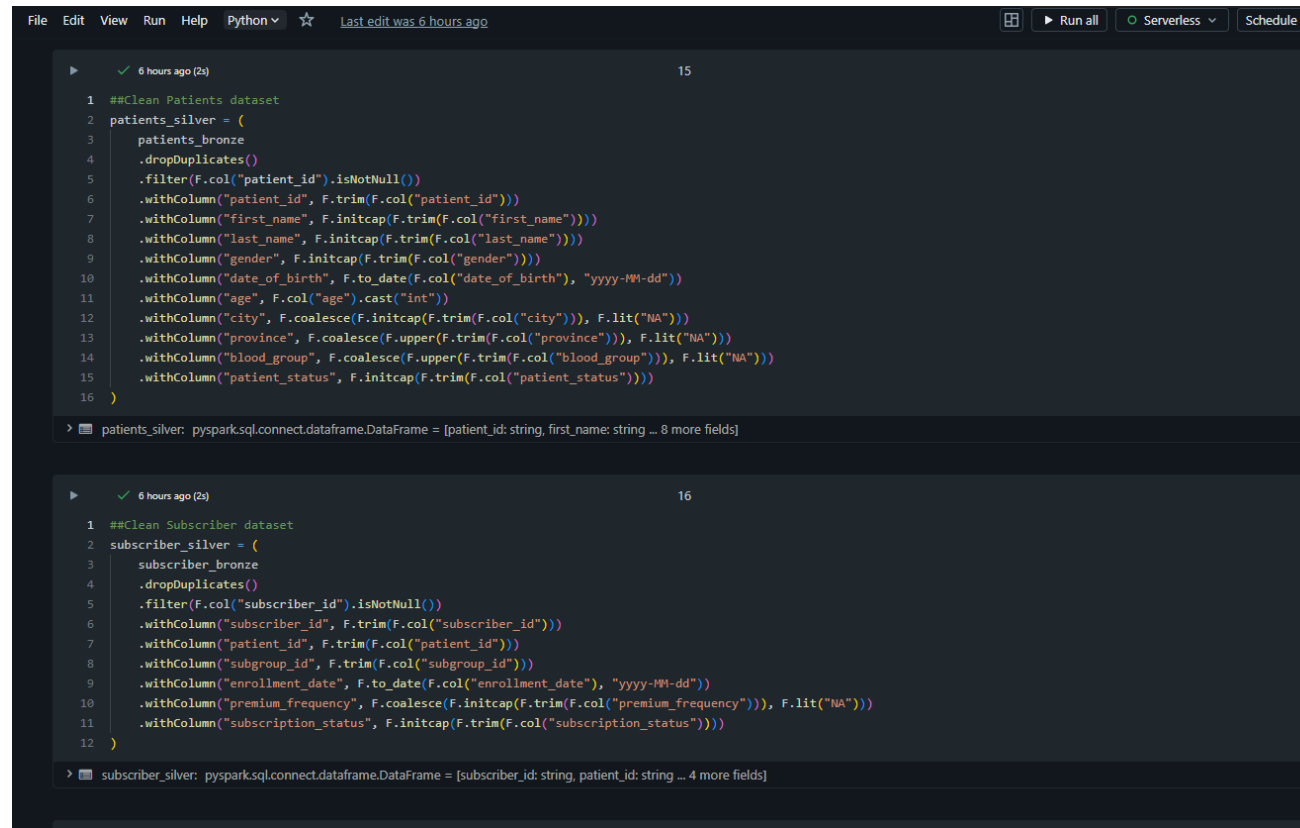
patients duplicate count: 2
subscriber duplicate count: 2
claims duplicate count: 3
group_subgroup duplicate count: 0

15
1
```

Figure 12. Duplicate Record Checks Performed

Step 13: Silver Transformation Logic Applied

- PySpark transformations were applied to clean and standardize the Bronze datasets. The cleaning included trimming spaces, standardizing text case, handling null values, casting data types, converting dates, and removing duplicate records.



```
File Edit View Run Help Python ☆ Last edit was 6 hours ago [Grid] ▶ Run all Serverless Schedule

▶ 6 hours ago (2s) 15
1 ##Clean Patients dataset
2 patients_silver = (
3     patients_bronze
4     .dropDuplicates()
5     .filter(F.col("patient_id").isNotNull())
6     .withColumn("patient_id", F.trim(F.col("patient_id")))
7     .withColumn("first_name", F.initcap(F.trim(F.col("first_name"))))
8     .withColumn("last_name", F.initcap(F.trim(F.col("last_name"))))
9     .withColumn("gender", F.initcap(F.trim(F.col("gender"))))
10    .withColumn("date_of_birth", F.to_date(F.col("date_of_birth"), "yyyy-MM-dd"))
11    .withColumn("age", F.col("age").cast("int"))
12    .withColumn("city", F.coalesce(F.initcap(F.trim(F.col("city"))), F.lit("NA")))
13    .withColumn("province", F.coalesce(F.upper(F.trim(F.col("province"))), F.lit("NA")))
14    .withColumn("blood_group", F.coalesce(F.upper(F.trim(F.col("blood_group"))), F.lit("NA")))
15    .withColumn("patient_status", F.initcap(F.trim(F.col("patient_status"))))
16 )

> patients_silver: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, first_name: string ... 8 more fields]

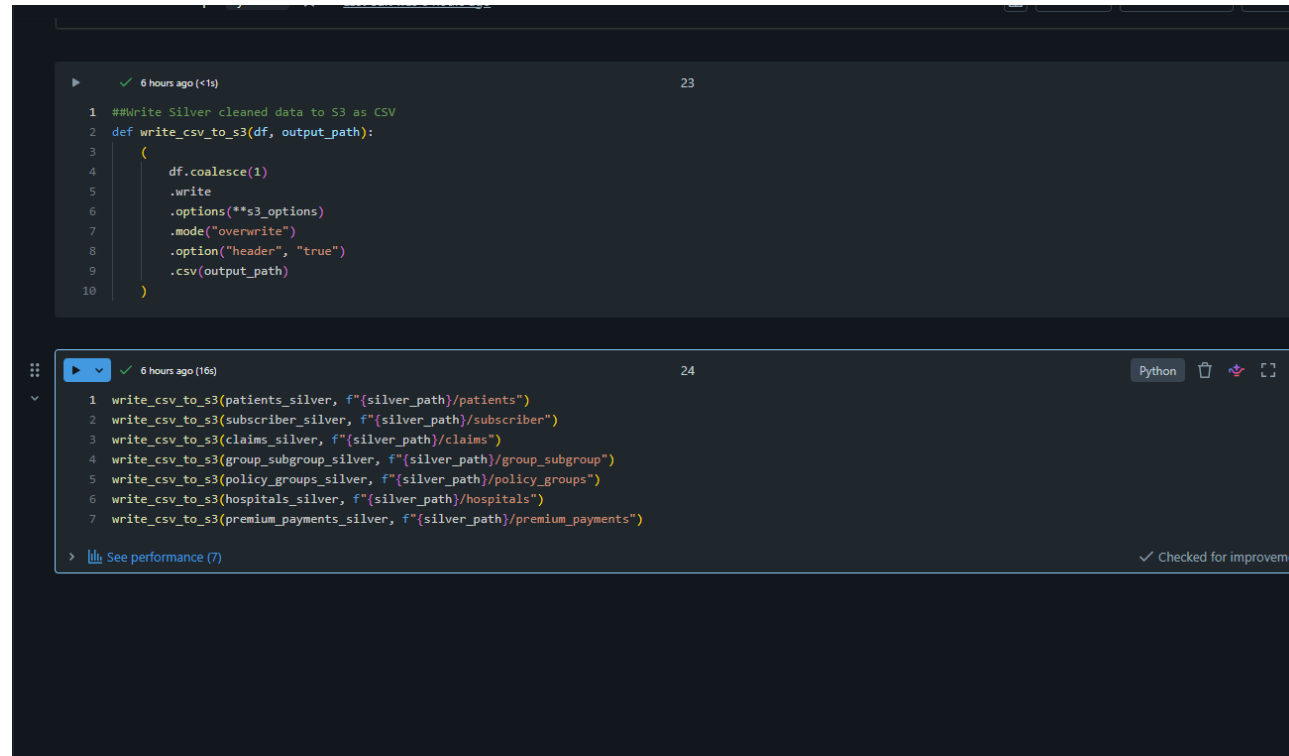
▶ 6 hours ago (2s) 16
1 ##Clean Subscriber dataset
2 subscriber_silver = (
3     subscriber_bronze
4     .dropDuplicates()
5     .filter(F.col("subscriber_id").isNotNull())
6     .withColumn("subscriber_id", F.trim(F.col("subscriber_id")))
7     .withColumn("patient_id", F.trim(F.col("patient_id")))
8     .withColumn("subgroup_id", F.trim(F.col("subgroup_id")))
9     .withColumn("enrollment_date", F.to_date(F.col("enrollment_date"), "yyyy-MM-dd"))
10    .withColumn("premium_frequency", F.coalesce(F.initcap(F.trim(F.col("premium_frequency"))), F.lit("NA")))
11    .withColumn("subscription_status", F.initcap(F.trim(F.col("subscription_status"))))
12 )

> subscriber_silver: pyspark.sql.connect.dataframe.DataFrame = [subscriber_id: string, patient_id: string ... 4 more fields]
```

Figure 13. Silver Transformation Logic Applied

Step 14: Cleaned Silver Data Written to S3 as CSV

- Nulls were replaced with "NA" where appropriate, duplicate rows were dropped, text fields were trimmed and standardised, and date columns were cast to a proper date type.



```
1 ##Write Silver cleaned data to S3 as CSV
2 def write_csv_to_s3(df, output_path):
3     (
4         df.coalesce(1)
5         .write
6         .options(*s3_options)
7         .mode("overwrite")
8         .option("header", "true")
9         .csv(output_path)
10    )

1 write_csv_to_s3(patients_silver, f"{silver_path}/patients")
2 write_csv_to_s3(subscriber_silver, f"{silver_path}/subscriber")
3 write_csv_to_s3(claims_silver, f"{silver_path}/claims")
4 write_csv_to_s3(group_subgroup_silver, f"{silver_path}/group_subgroup")
5 write_csv_to_s3(policy_groups_silver, f"{silver_path}/policy_groups")
6 write_csv_to_s3(hospitals_silver, f"{silver_path}/hospitals")
7 write_csv_to_s3(premium_payments_silver, f"{silver_path}/premium_payments")
```

Figure 14. Cleaned Silver Data Written to S3 as CSV

Step 15: Silver Layer Verified in S3

- The Silver layer was verified in S3 and contains cleaned dataset folders. These Silver outputs are the trusted data source for Redshift loading and analytical processing.

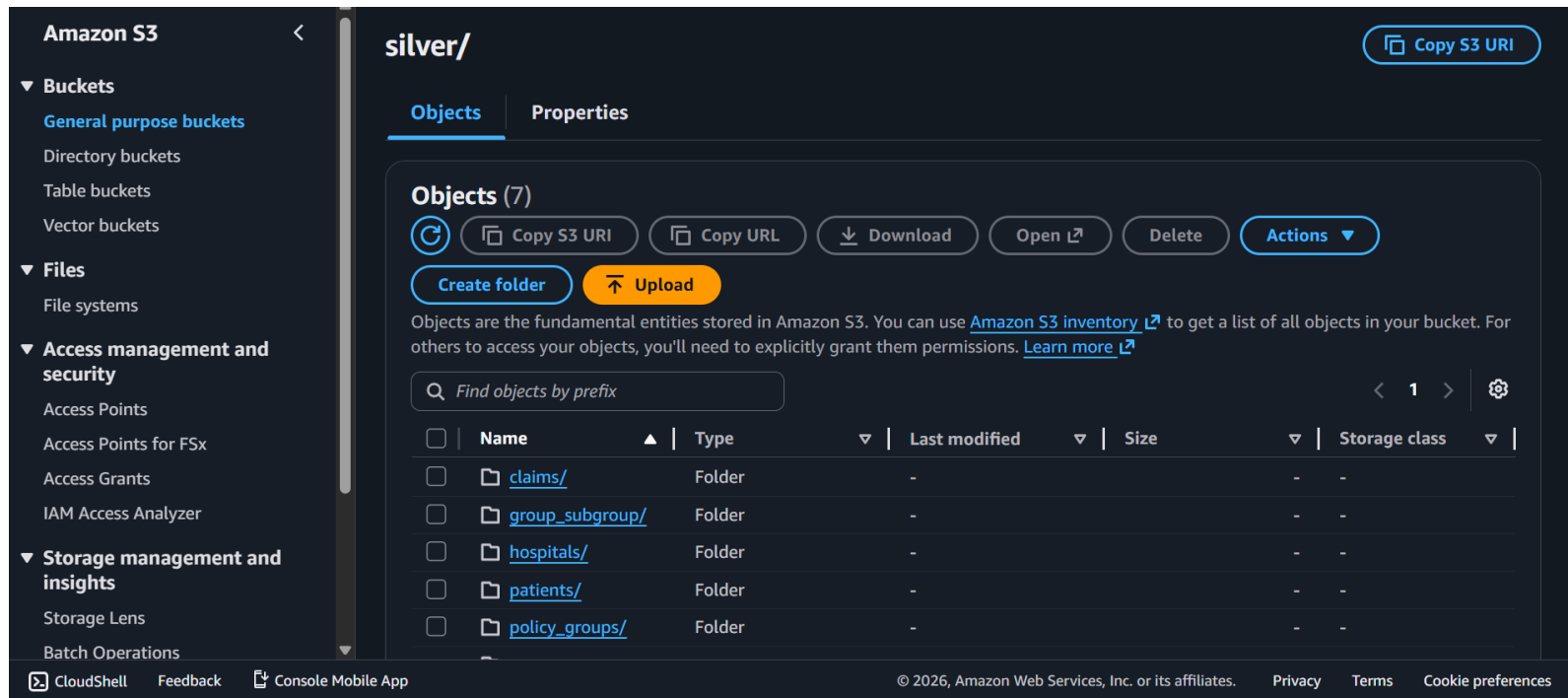


Figure 15. Silver Layer Verified in S3

Step 16: Jira Board Updated During Implementation

- The Jira board was updated as implementation progressed. Completed documentation tasks were moved to Done while Redshift loading and output table tasks were tracked in progress.

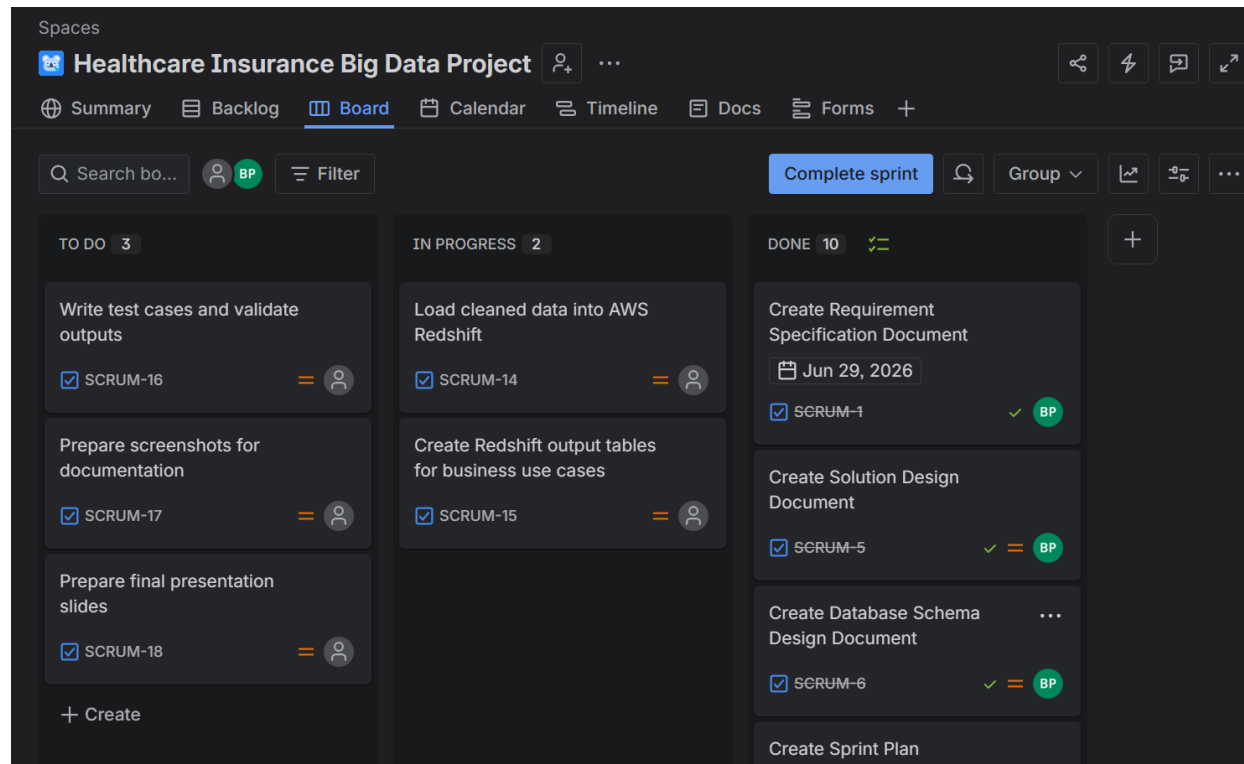


Figure 16. Jira Board Updated During Implementation

Step 17: Redshift Schemas and Silver Tables Created

- A clean schema was created in Redshift with a table for each of the four datasets, matching the column design from the Solution Design document.

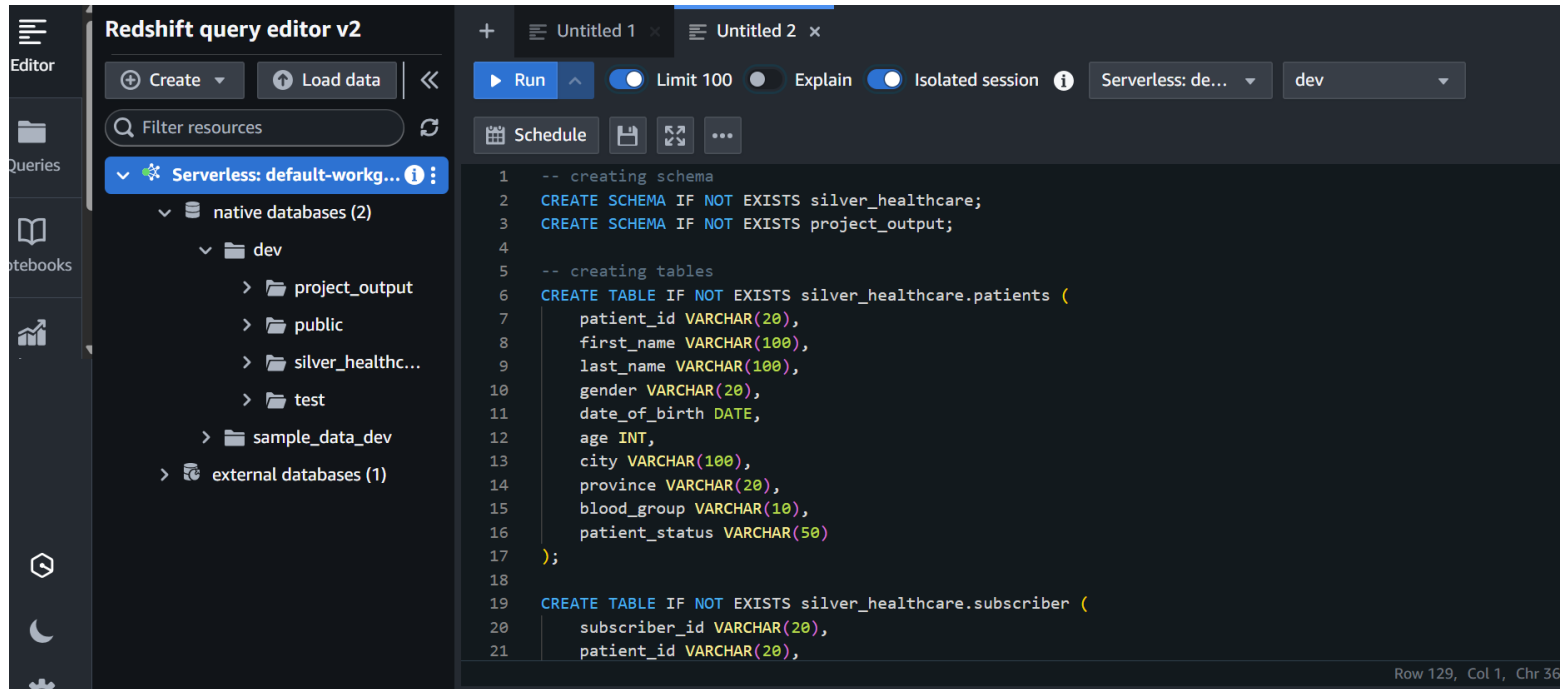


Figure 17. Redshift Schemas and Silver Tables Created

Step 18: COPY Commands Prepared for S3-to-Redshift Load

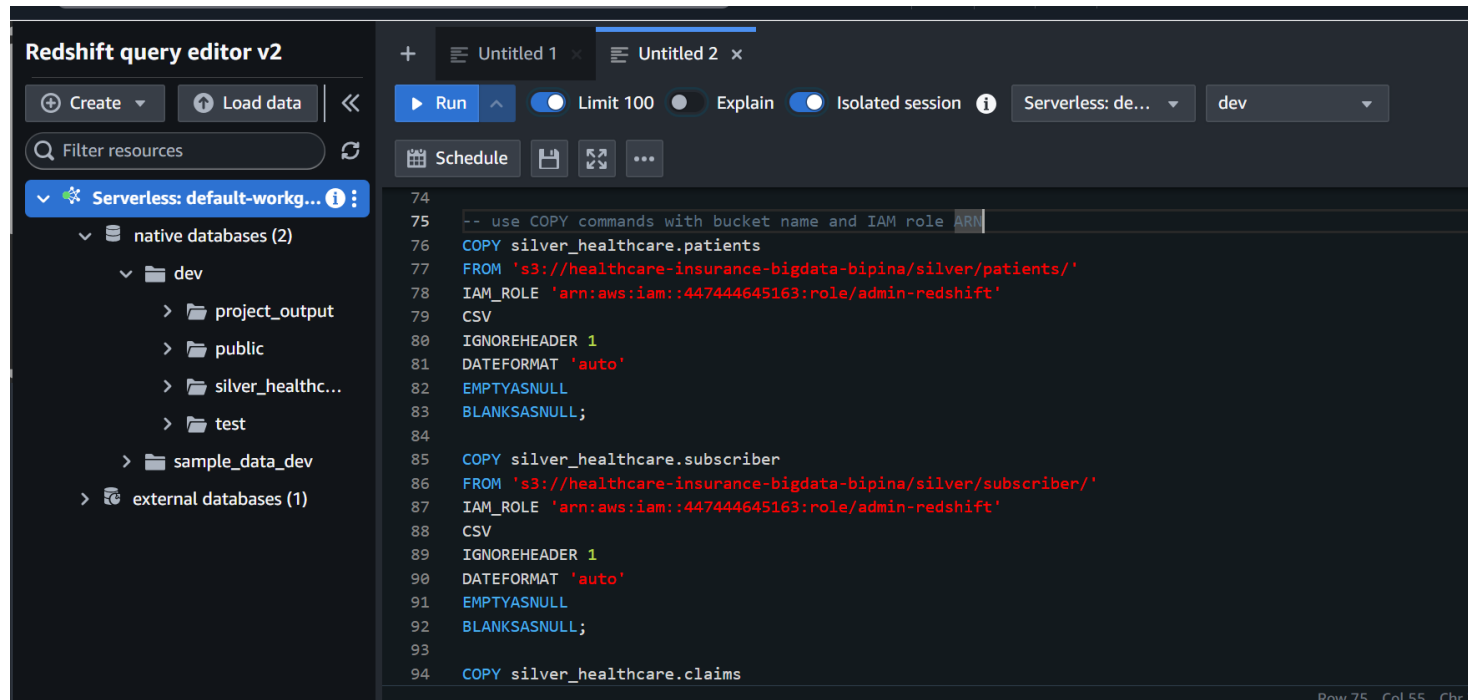


Figure 18. COPY Commands Prepared for S3-to-Redshift Load

Step 19: Redshift Load Validated in silver_healthcare Schema

-Row counts in Redshift were compared against the cleaned DataFrame counts to confirm nothing was lost or duplicated during the load.

The screenshot displays the Redshift query editor v2 interface. On the left, a sidebar shows the database structure under 'Serverless: default-workgroup' and 'native databases (2)'. The 'dev' database is expanded, showing folders for 'project_output', 'public', and 'silver_healthcare'. Under 'silver_healthcare', a 'Tables' folder is expanded, listing 'claims', 'group_subgroup', 'hospitals', 'patients', 'policy_groups', 'premium_payments', and 'subscriber'. The main editor area shows a SQL query with line numbers 138 to 147. The query is a multi-row SELECT statement with COUNT(*) for each table in the silver_healthcare schema. Below the query, the results are displayed in a table with two columns: 'count' and '183'. The status bar at the bottom indicates 'Elapsed time: 91802 ms' and 'Total rows: 1'.

```
138
139 --Validate Redshift load
140 SELECT COUNT(*) FROM silver_healthcare.patients;
141 SELECT COUNT(*) FROM silver_healthcare.subscriber;
142 SELECT COUNT(*) FROM silver_healthcare.claims;
143 SELECT COUNT(*) FROM silver_healthcare.group_subgroup;
144 SELECT COUNT(*) FROM silver_healthcare.policy_groups;
145 SELECT COUNT(*) FROM silver_healthcare.hospitals;
146 SELECT COUNT(*) FROM silver_healthcare.premium_payments;
147
```

count
183

Elapsed time: 91802 ms Total rows: 1

Figure 19. Redshift Load Validated in silver_healthcare Schema

Step 20: Final project_output Tables Created

Purpose	Description
Final project_output Tables Created	Final business output tables were created in Redshift under the project_output schema. These tables answer the required business questions from the capstone project.

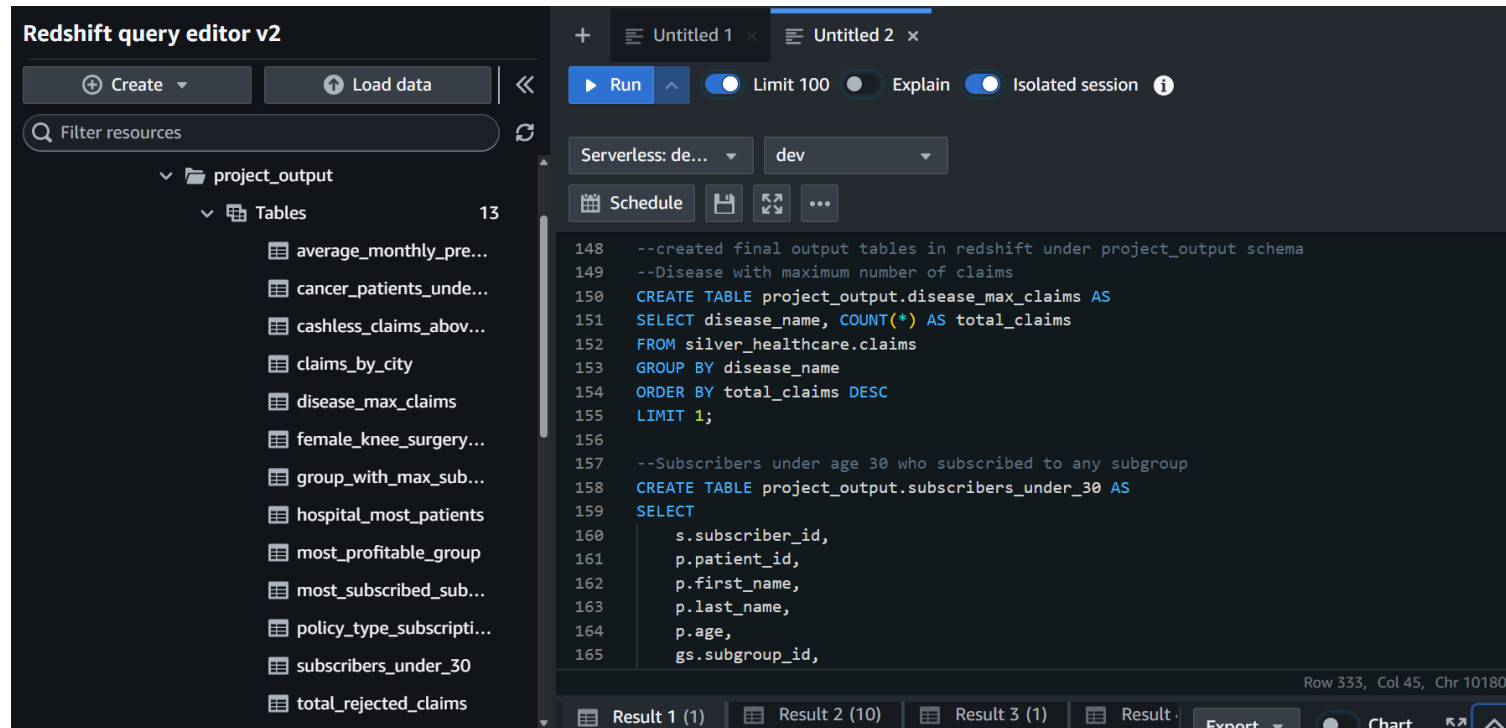


Figure 20. Final project_output Tables Created

Step 21: Output Tables Confirmed in Redshift

- A separate Project-Output schema was created, and each of the 13 business questions was turned into its own output table using PySpark or Redshift SQL.

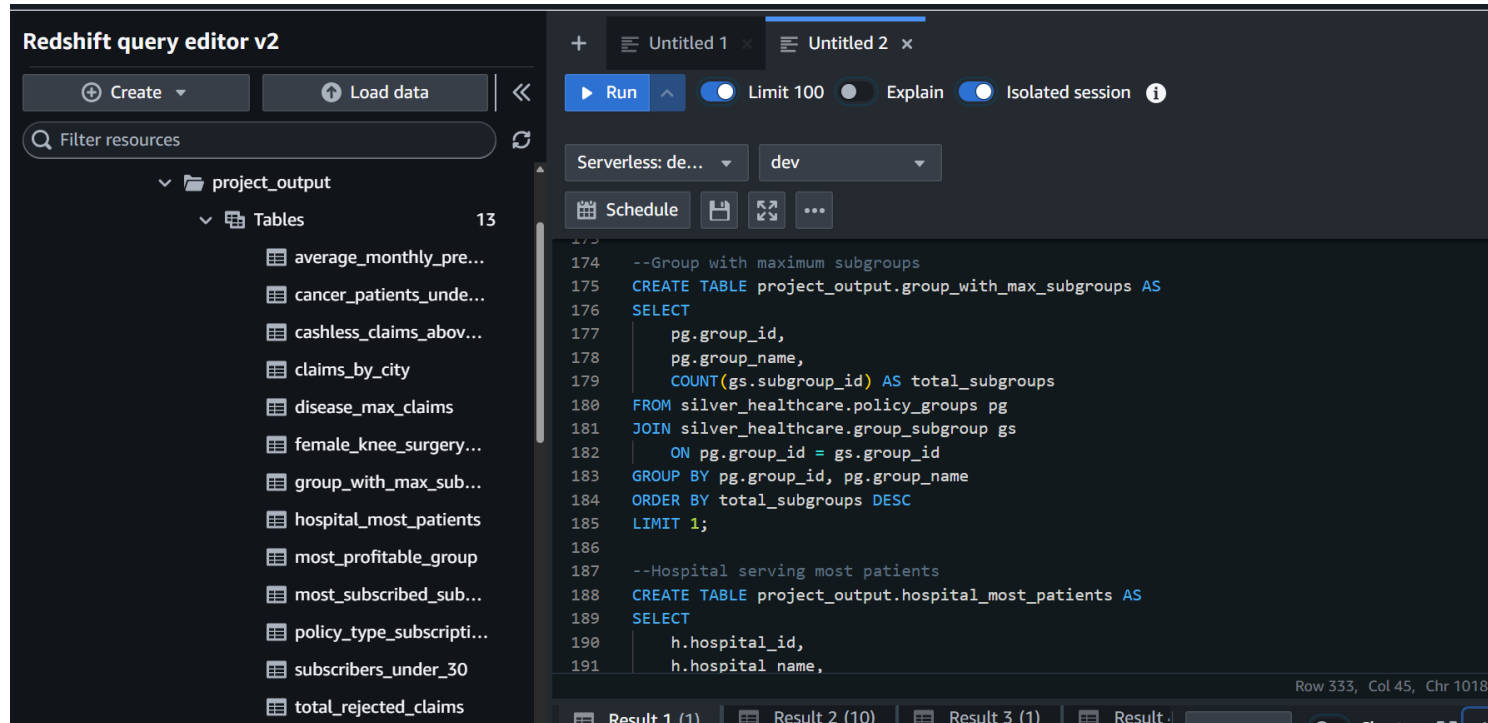


Figure 21. Output Tables Confirmed in Redshift

Step 22: Final Jira Dashboard Completed

- The final Jira board shows all project tasks completed and moved to Done. This confirms completion of the project planning, documentation, ingestion, transformation, Redshift loading, testing, screenshots, and presentation preparation tasks.

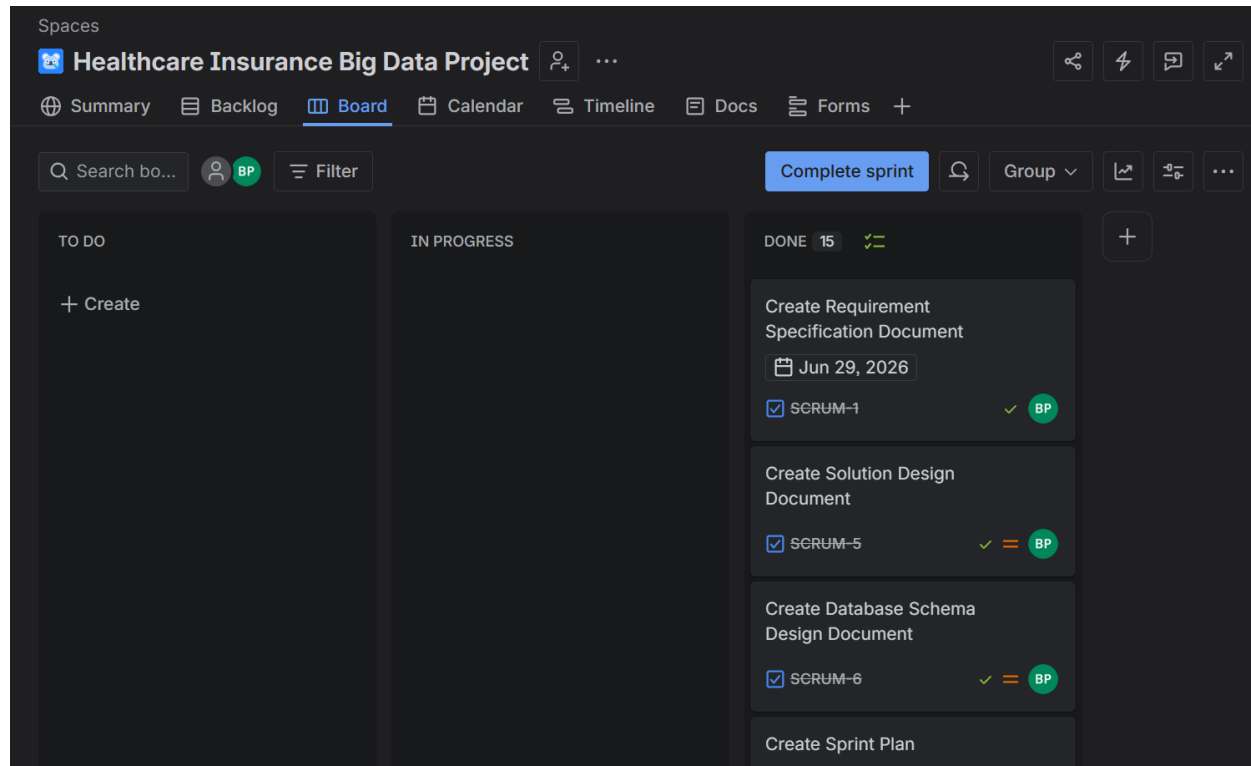


Figure 22. Final Jira Dashboard Completed

Conclusion

The project successfully demonstrates the complete implementation of a Healthcare Insurance Big Data Analytics pipeline from data ingestion to analytical reporting. Raw healthcare insurance datasets were initially stored in Amazon S3 and processed using PySpark within Databricks. During processing, the data was validated, cleaned, and organized into Bronze and Silver layers to improve data quality and consistency. The cleaned datasets were then prepared for loading into Amazon Redshift, where business-oriented analytical queries were executed to generate the required output tables for all project use cases. Throughout the project lifecycle, Jira was used to manage tasks, monitor progress, and ensure that all planned activities were completed successfully.

Recommended files to upload with this evidence document:

- Requirement Specification Document
- Solution Design Document
- Database Schema Design Document
- Databricks notebook or exported .ipynb file
- Redshift SQL scripts folder
- Input datasets or sample dataset folder
- This screenshot documentation file